



Inspiring Excellence

CSE370 : Database Systems

Project Report

Project Title : BRACU Club Management System (BRACU CMS)

Group No : 14 , CSE370 Lab Section : 13 , Summer 2026		
ID	Name	Contribution
23201623	S. M. Abrar Shaleheen	Features 1–3: smart club matching, recruitment pipeline, event proposal with venue clash detection. EER diagram, authentication, student & recruitment UI.
23201521	Abdus Rafi Sami	Features 4–6: task & workload analysis, budget approval with audit trail, collaboration board with notifications. Schema diagram, normalization, admin & analytics UI.

Table of Contents

Section No	Content	Page No
1	Introduction	3
2	Project Features	3
3	ER/EER Diagram	5
4	Schema Diagram	6
5	Normalization	7
6	Frontend Development	10
7	Backend Development	13
8	Source Code Repository	14
9	Conclusion	14
10	References	15

1. Introduction

BRAC University runs a large number of student clubs under the Office of Co-curricular Activities (OCA). At present most of the work around those clubs is done informally recruitment through social-media posts and spreadsheets, event and venue booking through e-mail chains, budget approval through printed forms, and task assignment through group chats. Because the information lives in disconnected places, students cannot easily find the club that actually suits them, two clubs can unknowingly book the same venue for the same slot, budget decisions leave no traceable record, and executives have no visibility of how much work each member is already carrying.

The BRACU Club Management System (BRACU CMS) is a database-driven web application that brings all of these activities into one relational database and one interface. It supports three roles. A student can build a profile of skills and interests, receive ranked club recommendations, apply to open recruitment campaigns with a CV, register for events and track personal tasks. A club executive can run recruitment pipelines, propose events with automatic venue clash detection, submit itemised budget requests, assign tasks while monitoring member workload, and post or answer inter-club collaboration requests. The OCA administrator gets a campus-wide view: approving events and budgets, managing clubs, venues and users, publishing announcements and reading analytics.

The system is implemented as a plain PHP application talking to a MySQL/MariaDB database through PDO prepared statements, with vanilla JavaScript (fetch/AJAX) for the interactive parts. The database consists of 25 normalised tables covering users and profiles, clubs and memberships, the skill/interest taxonomy, recruitment, events and venues, budgets, tasks, collaborations, notifications and audit logs, all connected by explicit foreign keys with appropriate ON DELETE behaviour.

2. Project Features

ID, Name	Features [3 per member]
23201623,S. M. Abrar Shaleheen	Ft 1: Skill & Interest Based Club Matching: A student records skills and interests in the profile; every active club records the skills it seeks and the interest areas it focuses on. The system compares the two sets and produces a percentage match (60% weight on skill overlap, 40% on interest overlap) together with human-readable reasons, then ranks all clubs for that student on the discovery page. Ft 2 : Digital Recruitment & Application Pipeline: Executives publish recruitment campaigns with a number of positions and a deadline. Students apply online with motivation, experience, portfolio link and a CV upload. Each application moves through a six-stage pipeline (submitted → under review → shortlisted → interview → accepted / rejected) using a drag-and-drop board, and reviewers can attach private notes to any applicant. Ft 3 : Event Proposal with Venue Clash Detection: While an executive fills the event proposal form, the chosen venue, date and time window are checked against every submitted/approved event in real time (AJAX). Overlapping bookings raise an immediate conflict warning, university-wide major events on the same date raise a second warning,

	and the system suggests alternative venues that are free in that exact slot. The OCA administrator then approves or rejects with a comment.
23201521,Abdus Rafi Sami	Ft 4 : Task Assignment & Member Workload Analysis: Executives create tasks with an estimated workload, priority and deadline. For every member the system computes a weighted workload score (estimated workload × priority weight × deadline urgency weight) against a fixed capacity and classifies the member as Healthy, Busy or Overloaded. Reassigning a task to an already overloaded member triggers a confirmation prompt, so work can be redistributed to members with spare capacity. Ft 5 : Budget Request, Review & Audit Trail: Clubs submit itemised budget requests (category-wise line items) linked to an event. The OCA administrator reviews, approves a possibly reduced amount, or rejects with a comment. Every state change draft, submitted, under review, approved, rejected, is written to an immutable audit log with the actor and timestamp, and the full history can be expanded from the request card. Ft 6 : Inter-Club Collaboration Board & Notification System: A club posts a collaboration request (equipment, human resource, co-host, venue, media) with a priority and a required date; other clubs browse the board and respond with an offer, which the requesting club accepts or declines. Every important action in the system application status change, event decision, budget decision, task assignment, collaboration response generates an in-app notification for the affected user, alongside campus-wide announcements and role-based analytics dashboards.

3. ER/EER Diagram

Drawn with draw.io. USER is a superclass with a disjoint specialisation into STUDENT, EXEC and ADMIN; the model also contains CLUB, SKILL, INTEREST, EVENT, VENUE, RECRUITMENT CAMPAIGN, APPLICATION, BUDGET REQUEST, BUDGET ITEM, TASK, COLLABORATION REQUEST/RESPONSE, NOTIFICATION, ANNOUNCEMENT and the audit logs, with all M:N relationships (HAS_SKILL, HAS_INTEREST, SEEKS_SKILL, FOCUSES_ON, MEMBER_OF, REGISTERS_FOR) and cardinality constraints shown.

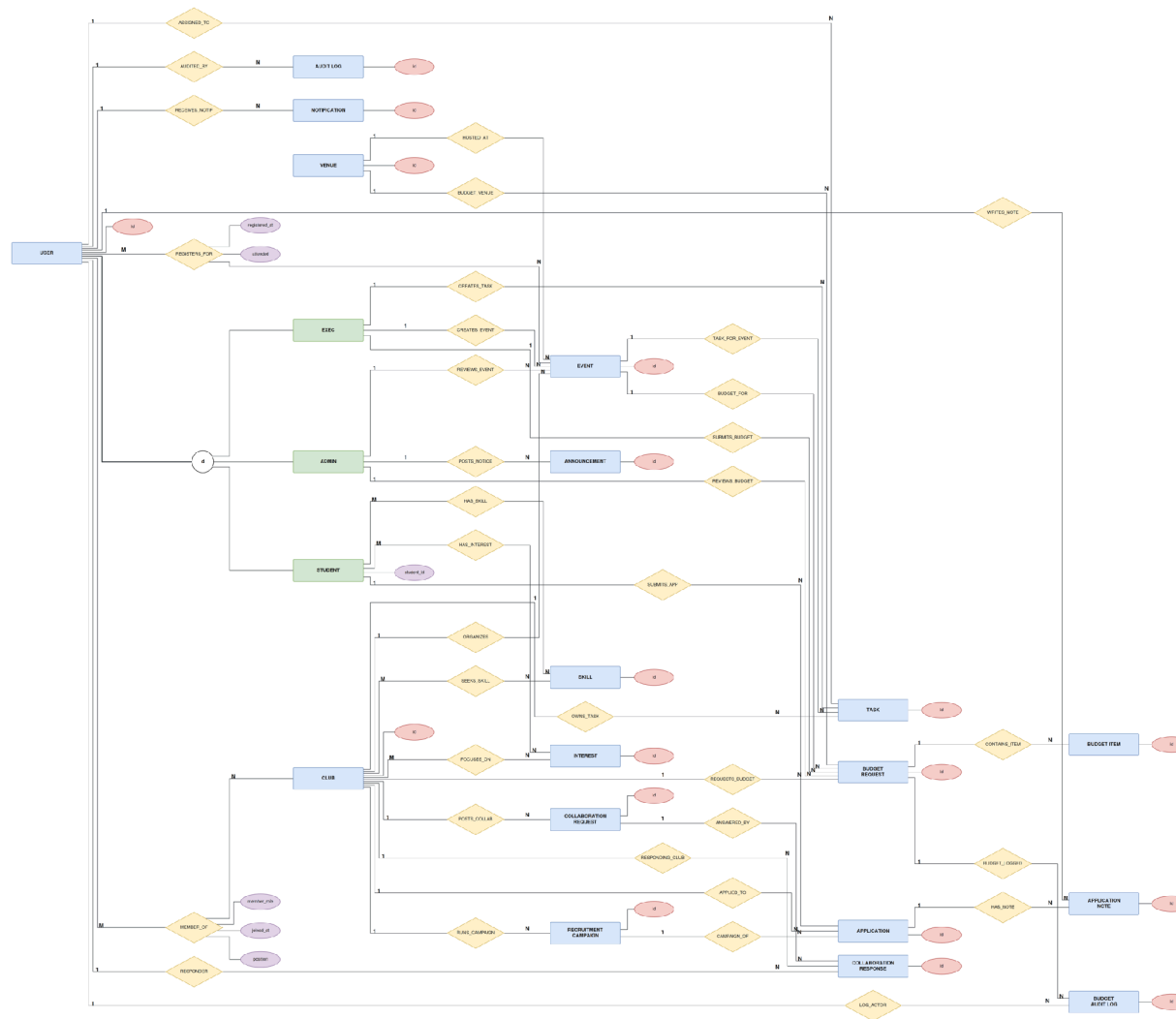


Figure 1: EER Diagram of BRACU CMS (drawn in draw.io)

4. Schema Diagram

Direct mapping of the EER diagram, drawn in draw.io. Primary keys are green (underlined), foreign keys yellow, and each arrow points from a foreign key to the primary key it references. The USER specialisation was mapped to one users table with a role attribute plus student_profiles for student-only attributes; every M:N relationship became its own relation with a composite primary key.



Figure 2: Schema Diagram of BRACU CMS (drawn in draw.io)

5. Normalization

a. First Normal Form (1NF)

The converted schema is in 1NF. Every relation has a declared primary key, there are no repeating groups and no composite or multi-valued attributes are stored in a single column.

- The multi-valued facts "a student has many skills / interests" and "a club seeks many skills / focuses on many interest areas" were not stored as comma-separated lists. They were decomposed at design time into the relations `student_skills(user_id, skill_id)`, `student_interests(user_id, interest_id)`, `club_skills(club_id, skill_id)` and `club_interests(club_id, interest_id)`, each with a composite primary key, plus the lookup relations `skills(id, name)` and `interests(id, name)`.
- The itemised cost breakdown of a budget request is likewise a repeating group and was decomposed into `budget_items(id, budget_request_id, category, amount)` instead of being packed into one column of `budget_requests`.
- Attributes such as `applications.motivation`, `applications.experience`, `events.required_equipment` and `clubs.description` are long text fields, but each holds one single descriptive value about the row and is never decomposed or searched component-wise, so they remain atomic with respect to the application.

Therefore no decomposition was needed to reach 1NF.

b. Second Normal Form (2NF)

The schema is in 2NF: it is already in 1NF, and no non-prime attribute is partially dependent on a proper subset of a candidate key.

- Most relations (`users`, `clubs`, `events`, `applications`, `tasks`, `budget_requests`, `collaboration_requests`, `notifications`, ...) use a single-attribute surrogate primary key `id`. A partial functional dependency is impossible on a single-attribute key, so these relations satisfy 2NF automatically.
- The relations that do have a composite key, `student_skills`, `student_interests`, `club_skills`, `club_interests`, are pure relationship (bridge) relations that contain no non-prime attributes at all. With nothing outside the key, there is nothing that could depend on part of it, so they are also in 2NF.
- `club_members` and `event_registrations` deserve special mention. Both carry a surrogate id together with a UNIQUE constraint on (`club_id`, `user_id`) and (`event_id`, `user_id`) respectively, so the pair is a candidate key. Their descriptive attributes, `position`, `member_role`, `status`, `joined_at`, and `registered_at`, `attended`, describe the combination of the two entities and not either one alone: `position` depends on which member is in which club, not on the club alone or the student alone. Hence there is no partial dependency here either.

A potential partial dependency would have appeared if, for example, `club_members` also stored the member's name or e-mail (which depends on `user_id` alone) or the club's category (which depends on `club_id` alone). Those attributes were deliberately kept in `users` and `clubs` and are fetched through a join, so the schema needed no decomposition for 2NF.

c. Third Normal Form (3NF)

Almost all relations of the converted schema are in 3NF: no non-prime attribute is transitively dependent on the primary key, because descriptive data about a referenced entity is never copied into the referencing relation.

- events stores only venue_id, not the venue name, capacity or location; those depend on venues.id and are obtained by a join, so no transitive dependency $id \rightarrow venue_id \rightarrow venue_name$ exists.
- applications stores club_id, user_id and campaign_id but not the club name, the applicant name or the campaign title; tasks stores assigned_to and club_id but not the assignee's name; notifications stores user_id but not the recipient's e-mail. In each case the dependent data stays in the relation of the entity it belongs to.
- budget_items(id, budget_request_id, category, amount) has only attributes of the line item itself, and budget_audit_log keeps only the request id, the action, the actor id and the timestamp.

One relation does contain a transitive dependency, and it is an intentional design decision rather than an oversight. In budget_requests the attributes event_name, event_date, venue_id and expected_attendance duplicate data that is also reachable through event_id, giving the dependency chain $id \rightarrow event_id \rightarrow event_name / event_date$. Strictly, a lossless decomposition into

budget_requests(id, event_id, club_id, requested_amount, approved_amount, justification, status, submitted_by, submitted_at, reviewed_by, reviewed_at, review_comment)

events(id, title, event_date, venue_id, expected_attendance, ...)

would remove it, and the duplicated columns could then be recovered with a join on event_id. The columns were nevertheless retained for two reasons. First, event_id is nullable and declared ON DELETE SET NULL, because a club may request a budget for an activity that has not yet been entered as an event, or the linked event may later be cancelled; in those rows the values are supplied independently and are not derivable from any other relation, so the dependency does not actually hold. Second, an approved budget is a financial record that must keep the event details exactly as they were at the moment of approval, even if the event is afterwards edited; the copied columns act as a deliberate historical snapshot for the audit trail. Apart from this documented exception, the schema is in 3NF, and because every determinant in the remaining relations is a candidate key, those relations are in BCNF as well.

The normalized schema is shown in Figure 3 on the following page.

Normalized Schema (3NF)

Final normalized relational schema of BRACU CMS after the 1NF / 2NF / 3NF analysis above: 25 relations in 3NF, primary keys in green, foreign keys in yellow.



Figure 3: Normalized Schema (3NF) of BRACU CMS (drawn in draw.io)

6. Frontend Development

The frontend is written in HTML5, a hand-built CSS design system (assets/css/style.css and pages.css CSS custom properties, a responsive card/grid layout, badges, modals and a role-aware sidebar) and vanilla JavaScript. There is no framework and no build step: every interactive action calls a JSON endpoint under /api/ with fetch() through the helpers in assets/js/ajax.js, and the affected part of the page is updated in place. Dashboard charts are rendered with Chart.js through thin wrappers in assets/js/charts.js. The interface is shared by three navigation shells: nav_student.php, nav_exec.php and nav_admin.php, which are included by a common layout so that all pages stay visually consistent.

Contribution of ID : 23201623, Name : S. M. Abrar Shaleheen

Built the shared layout and CSS design system, the login and registration pages, and the student-facing interface: the profile editor with skill/interest chips and CV upload, the club discovery page (student/clubs.php) where each club card shows an animated match-percentage ring with the list of matching reasons, the club detail page, the application form and the "my applications" status tracker. On the executive side he built the recruitment board (exec/recruitment.php) where applicant cards are dragged between the six pipeline columns and the status is persisted over AJAX, and the event proposal form (exec/events.php) whose venue, date and time inputs fire a live conflict check and render the warning banner and the suggested alternative venues without a page reload.

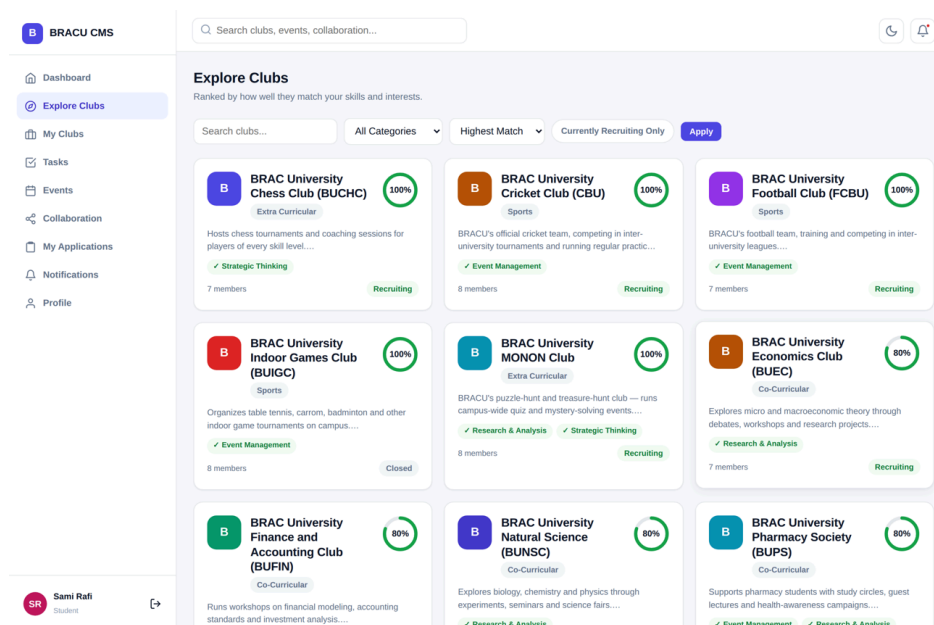


Figure 4: student/clubs.php clubs ranked by match percentage with the matched-skill chips (Ft 1)

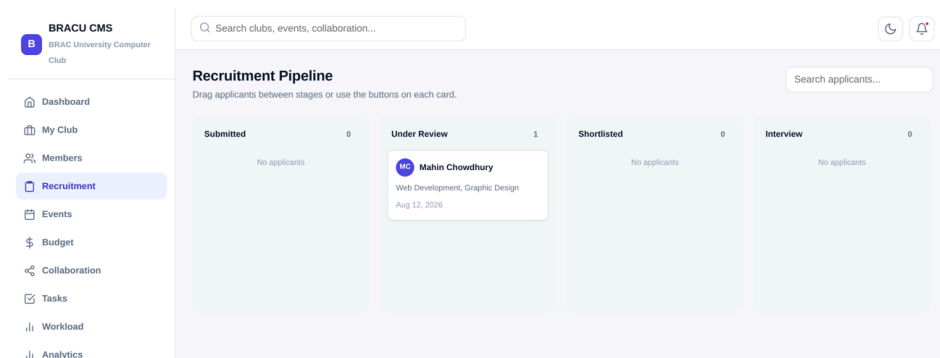


Figure 5: `exec/recruitment.php` drag-and-drop recruitment pipeline (Ft 2)

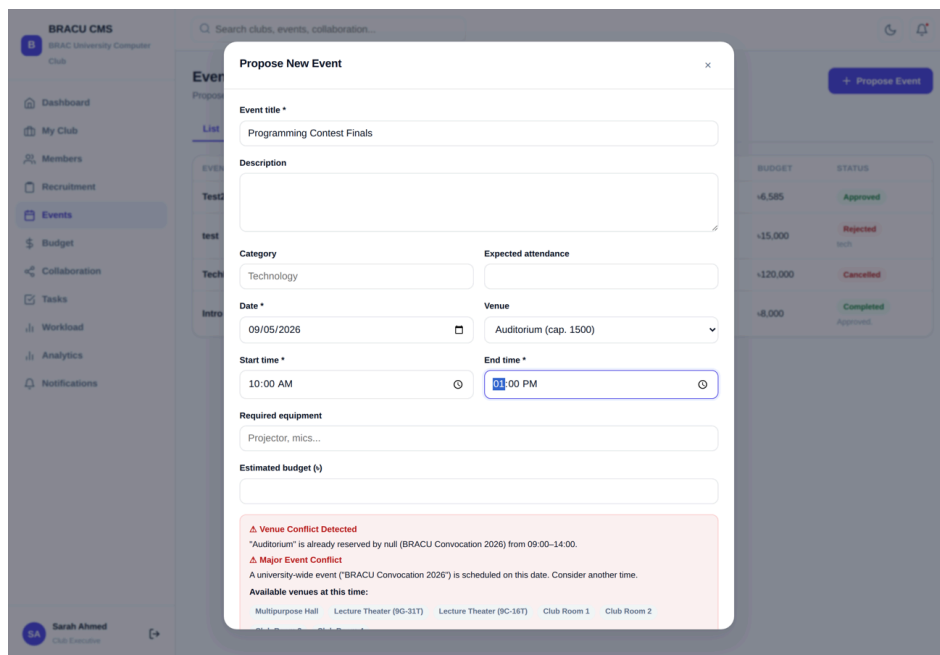


Figure 6: `exec/events.php` venue conflict and major-event warnings with suggested free venues (Ft 3)

Contribution of ID : 23201521, Name : Abdus Rafi Sami

Built the executive management and administrator interfaces: the task board (`exec/tasks.php`) with inline status changes and the reassignment modal that warns before moving a task to an overloaded member, the workload analysis page (`exec/workload.php`) with per-member capacity bars and Healthy/Busy/Overloaded badges, the budget page (`exec/budget.php`) with the dynamic line-item builder and the expandable audit-trail timeline, and the collaboration board (`exec/collaboration.php`) with its browse-and-respond flow. He also built the whole admin section: approval queues for events and budgets, club, user and venue management, announcements plus the notification centre and the Chart.js analytics dashboards used by both executives and the OCA administrator.

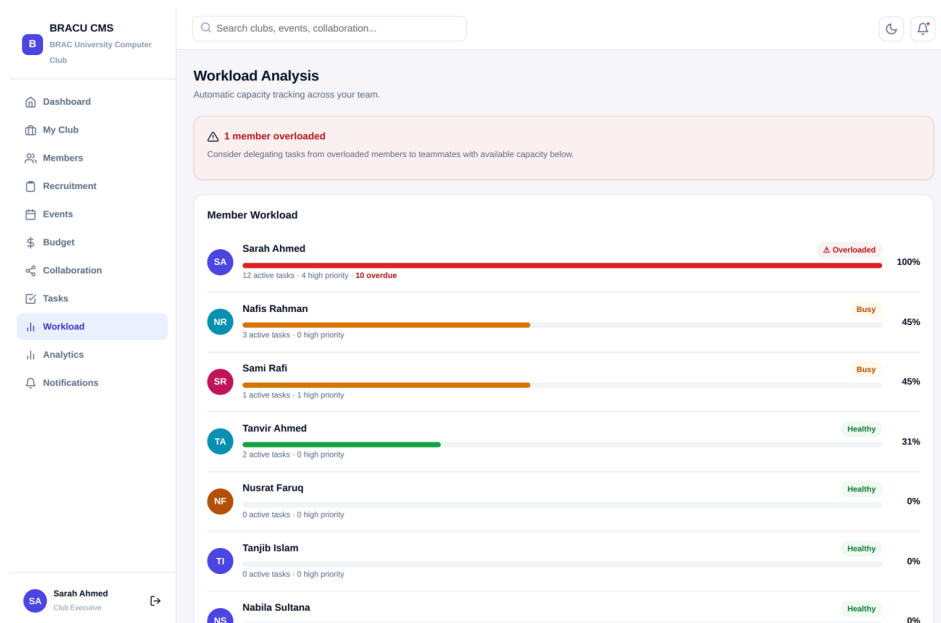


Figure 7: `exec/workload.php` capacity bars with Healthy / Busy / Overloaded classification (Ft 4)

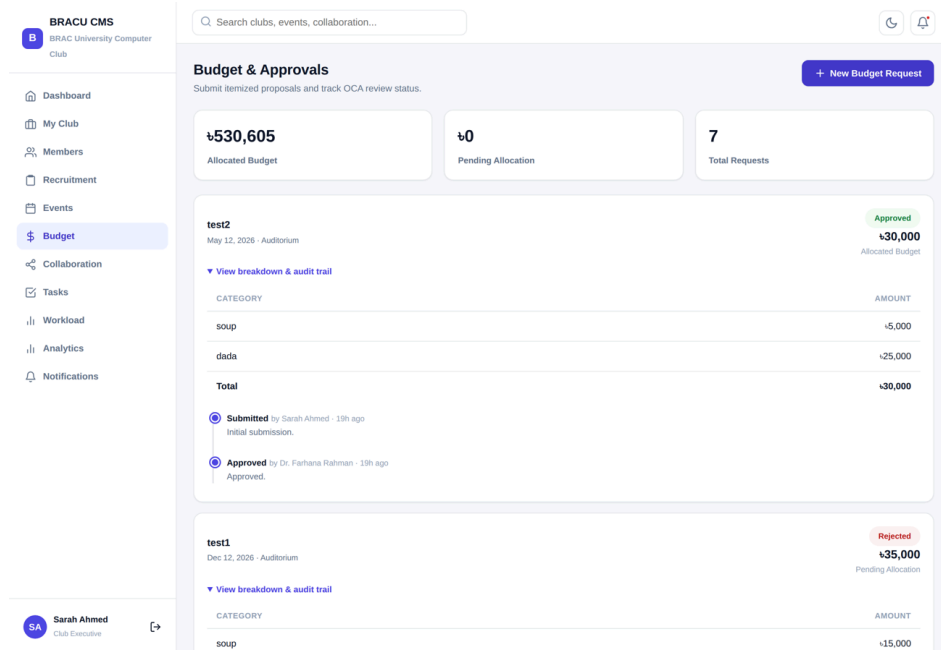


Figure 8: `exec/budget.php` itemised breakdown with the expanded audit trail (Ft 5)

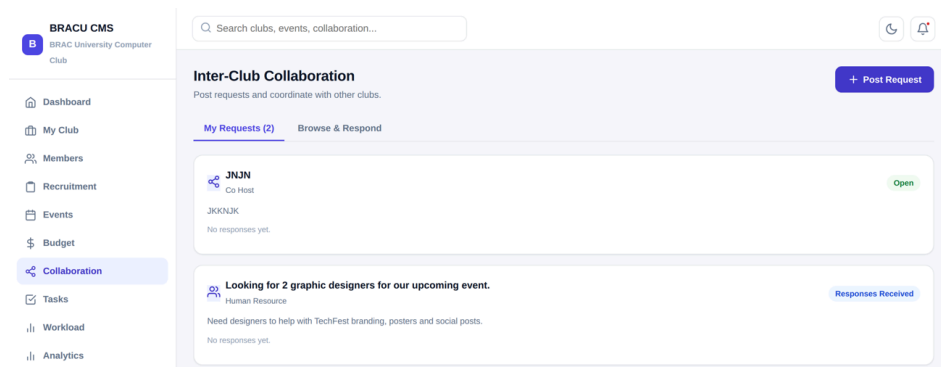


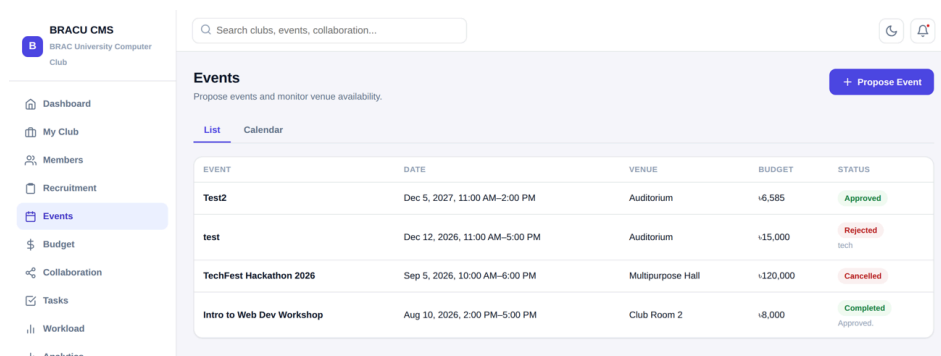
Figure 9: `exec/collaboration.php` inter-club collaboration board (Ft 6)

7. Backend Development

The backend is plain PHP (no framework) on Apache/XAMPP with a MySQL/MariaDB database named `bracu_cms`. A single PDO connection is created in `config/db.php` with exceptions enabled and emulated prepares turned off, and every query in the project without exception is a prepared statement with bound parameters, which removes SQL injection. `includes/auth.php` handles session login, password hashing with `password_hash()/password_verify()`, the role guards `requireLogin()`, `requireExec()` and `requireAdmin()`, and CSRF token generation; every state-changing endpoint in `/api/` starts with the same bootstrap that verifies the session, the role and the CSRF token before touching the database, and replies with JSON. Shared domain logic lives in `includes/functions.php`, and cross-cutting concerns are handled by the helpers `notify()` (writes a row into notifications) and `logAudit()` (writes into `audit_log`).

Contribution of ID : 23201623, Name : S. M. Abrar Shaleheen

- Database connection layer (`config/db.php`), session/authentication/authorization layer (`includes/auth.php`), CSRF verification and the JSON response helpers.
- Ft 1: `computeClubMatch()` and `getAllClubMatches()`: joins `student_skills/student_interests` against `club_skills/club_interests`, computes the overlap ratios, weights them 60/40, builds the reason list and sorts every active club by match score.
- Ft 2: the recruitment endpoints: `create_campaign.php`, `toggle_campaign.php`, `apply.php` (validates the application, stores it with status submitted and notifies the club), `upload_cv.php` (MIME/size validated PDF upload into `uploads/cv/` with a randomised filename), `update_application_status.php` (moves an application along the pipeline inside a transaction and notifies the applicant) and the reviewer-note insert into `application_notes`.
- Ft 3: the clash-detection engine: `checkVenueConflict()` detects an interval overlap with the predicate `NOT (end_time <= ? OR start_time >= ?)` restricted to the same venue and date and to events in status submitted or approved; `checkMajorEventConflict()` flags university-wide events on the same date; `suggestAlternativeVenues()` returns every available venue that is free in that slot. These are exposed live to the browser through `api/check_conflict.php` and re-validated server-side in `api/create_event.php` and `api/event_status.php` so that a conflicting booking can never be written even if the client check is bypassed.



The screenshot shows the BRACU CMS interface. On the left is a sidebar with navigation links: Dashboard, My Club, Members, Recruitment, Events (highlighted), Budget, Collaboration, Tasks, Workload, and Analytics. The main content area is titled 'Events' and includes a search bar and a '+ Propose Event' button. Below this is a table with the following data:

EVENT	DATE	VENUE	BUDGET	STATUS
Test2	Dec 5, 2027, 11:00 AM-2:00 PM	Auditorium	+6,585	Approved
test	Dec 12, 2026, 11:00 AM-5:00 PM	Auditorium	+15,000	Rejected
TechFest Hackathon 2026	Sep 5, 2026, 10:00 AM-6:00 PM	Multipurpose Hall	+120,000	Cancelled
Intro to Web Dev Workshop	Aug 10, 2026, 2:00 PM-5:00 PM	Club Room 2	+8,000	Completed

Figure 10: `exec/events.php` event list reflecting the status written by `api/create_event.php` and `api/event_status.php`

Contribution of ID : 23201521, Name : Abdus Rafi Sami

- Wrote the full DDL script database/bracu_cms.sql: 25 tables with primary keys, foreign keys with the correct ON DELETE CASCADE / SET NULL behaviour, UNIQUE constraints (uniq_club_user, uniq_event_user, unique skill and interest names), ENUM-based status domains and the covering indexes idx_events_venue_date, idx_apps_club_status, idx_tasks_assignee_status and idx_notifications_user_read together with the demo seed data.
- Ft 4: computeMemberWorkload() and clubMembersWithWorkload(): aggregates every non-completed task of a member, weights each by priority (low 0.7, medium 1.0, high 1.5) and by deadline urgency (1.3 if overdue, 1.15 if due within three days), compares the total against the capacity constant and returns the percentage, the Healthy/Busy/Overloaded status and the counts of active, high-priority and overdue tasks. Endpoints create_task.php, update_task_status.php and reassign_task.php (which re-computes the target member's load before confirming).
- Ft 5: the budget workflow: submit_budget.php and submit_draft_budget.php insert the request and its budget_items rows in one transaction; review_budget.php applies the administrator decision, stores the approved amount and comment, appends a row to budget_audit_log and notifies the club executive, so the complete decision history of every request is reproducible.
- Ft 6: the collaboration endpoints create_collab.php, respond_collab.php and update_collab_status.php, the notification endpoints mark_notification_read.php and mark_all_read.php, the announcement and administrator endpoints (create_announcement.php, admin_club_action.php, admin_user_action.php, create_user.php, venue_action.php), and the aggregate SQL (GROUP BY / COUNT / SUM with joins) behind the analytics dashboards.

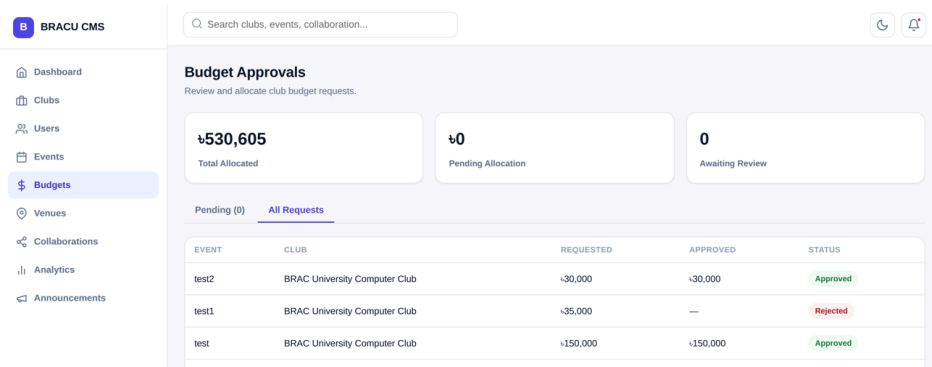


Figure 11: admin/budgets.php every request with its requested and approved amount after api/review_budget.php

8. Source Code Repository

The complete source code, the SQL dump (database/bracu_cms.sql) and the diagram files are available at the links below.

- Source code (GitHub) - <https://github.com/Assassin-Abrar/CSE370>
- EER Diagram- https://drive.google.com/file/d/1fCYvrQbeTSfZohXwGOyCpj4TUcKPJ0hE/view?usp=drive_link
- Schema Diagram - https://drive.google.com/file/d/1w4NB8sskG-ejzLMayxSTbXvfLRXJ8vrE/view?usp=drive_link

Setup: start Apache and MySQL in XAMPP, import database/bracu_cms.sql (it creates the bracu_cms database, all tables and the demo data), place the project folder in htdocs and open http://localhost/F_shitz/. All demo accounts use the password password123.

9. Conclusion

BRACU CMS demonstrates that the everyday administrative work of university clubs maps cleanly onto a well-designed relational database. Starting from an EER model of the domain, we mapped the specialisation of USER, resolved every many-to-many relationship into its own relation, and verified the result against 1NF, 2NF and 3NF, keeping only one documented and justified denormalisation in budget_requests. On top of that schema the application delivers six substantial features skill-based club matching, an online recruitment pipeline, venue clash detection, workload-aware task assignment, an auditable budget approval flow and an inter-club collaboration board each of which is driven by SQL rather than by hard-coded logic: matching is set overlap over bridge relations, clash detection is an interval-overlap predicate over an indexed (venue_id, event_date) pair, workload is a weighted aggregation over tasks, and the audit trail is an append-only relation.

The main limitations are that the application runs on a local XAMPP stack without HTTPS or e-mail delivery, that notifications are polled rather than pushed, and that the matching weights are fixed constants rather than learned from historical acceptance data. Natural extensions would be attendance tracking with QR codes, a semester-wise budget ceiling per club enforced at the database level, exportable OCA reports, and stored procedures or triggers to move parts of the notification and audit logic into the database itself.

10. References

- PHP Manual: PDO (prepared statements, transactions)
: <https://www.php.net/manual/en/book.pdo.php>
- PHP Manual: Password hashing (password_hash / password_verify)
: <https://www.php.net/manual/en/book.password.php>
- PHP Manual: File uploads: <https://www.php.net/manual/en/features.file-upload.php>
- MySQL 8.0 Reference Manual: <https://dev.mysql.com/doc/refman/8.0/en/>
- MySQL: FOREIGN KEY constraints and referential actions
: <https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html>
- MDN Web Docs :Fetch API
: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MDN Web Docs :HTML Drag and Drop API
: https://developer.mozilla.org/en-US/docs/Web/API/HTML_Drag_and_Drop_API
- MDN Web Docs :Using CSS custom properties
: https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties
- Chart.js v4 documentation : <https://www.chartjs.org/docs/latest/>
- OWASP : SQL Injection Prevention Cheat Sheet
: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP :Cross-Site Request Forgery Prevention Cheat Sheet
: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- draw.io (diagrams.net) :tool used for the EER and schema diagrams
: <https://app.diagrams.net/>

- Apache Friends :XAMPP for Windows :<https://www.apachefriends.org/index.html>
- W3Schools :SQL Tutorial (joins, aggregate functions) :<https://www.w3schools.com/sql/>
- Elmasri & Navathe, Fundamentals of Database Systems (7th ed.) :EER mapping and normalization (course text)
:<https://www.pearson.com/en-us/subject-catalog/p/fundamentals-of-database-systems/P200000003546>